

VisionGuard AI

Transforming Existing CCTV Cameras into Intelligent Security Systems

AI-Powered Railway Surveillance Intelligence Platform — UI/UX & MVP Architecture Generation Prompts (Google Stitch + Antigravity)

Problem Context

Indian Railways manages millions of passengers daily across a vast station and network footprint. The existing CCTV estate is used almost entirely for passive recording — footage is reviewed only after an incident, and live monitoring still depends on constant human attention. This creates delayed crime detection, unmanaged crowd congestion, passenger safety gaps, unmonitored workforce safety compliance, and unavoidable human error during long surveillance shifts. The objective of this submission is to convert Indian Railways' existing CCTV infrastructure into an intelligent, decision-support monitoring system using AI/ML — without requiring any camera hardware replacement.

VisionGuard AI is positioned as a real command-and-control platform for railway operations staff, not as an AI demo. Crowd intelligence, crime/incident detection, and workforce safety monitoring run continuously in the background as an intelligence layer; the visible product is a clean, data-first operations dashboard that a Railway Protection Force (RPF) control room would actually deploy.

At a Glance

Problem ID	SIH1349 — Ministry of Railways
Challenge	Using existing CCTV network for crowd management, crime prevention, and work monitoring using AI/ML
Product name	VisionGuard AI
Core pillars	Crowd Intelligence • Crime Prevention • Workforce Safety • Smart Alerts • Operational Analytics
Design mandate	Government enterprise control-room product — no AI marketing language, no robot/neon visual identity, no chatbot-style UI

Design Philosophy

AI is treated strictly as an internal intelligence layer, never the visual identity of the product. The interface must read as railway operations software that happens to be AI-powered underneath — not an AI product that happens to monitor railways.

AVOID

- Heavy AI branding or "AI revolution" style headings
- Robot illustrations / futuristic neon visuals
- ChatGPT-style conversational interfaces
- Marketing language over operational data

BUILD

- Government enterprise software appearance
- Railway control-room dashboard experience
- Clean, data-first, professional monitoring UI
- Operational decision-support, not experimentation

Google Stitch Prompt

Copy the full block below into Google Stitch as a single generation prompt. It defines the design language, the five required screens with exact content, and the shared component library so Stitch produces a coherent, government-grade dashboard rather than five disconnected screens.

PROMPT — PASTE INTO GOOGLE STITCH

ROLE & OBJECTIVE

Act as a senior enterprise UI/UX designer. Design a complete, high-fidelity UI/UX prototype for "VisionGuard AI" — an AI-powered CCTV intelligence platform for Indian

Railways (SIH1349, Ministry of Railways). This is a real government control-room product, not an AI demo. Do NOT use robot illustrations, neon gradients, chatbot-style layouts, or "AI revolution" marketing language anywhere in the design.

DESIGN LANGUAGE

- Style: enterprise dashboard / modern railway operations control room
- Tone: professional government technology interface, mission-critical and calm
- Theme: dark professional monitoring interface (deep navy / charcoal base, desaturated blues, single restrained accent color for alerts and status)
- Typography: clean, highly legible sans-serif; strong hierarchy between KPI numbers, section labels, and body text; numeric data should dominate visually
- Layout: data-first, grid-based, generous spacing between operational modules
- Motion: minimal, purposeful animation only (live-status pulses, alert badges) – no decorative motion
- Responsiveness: design for a large control-room monitor / desktop first, with a coherent tablet layout as a secondary breakpoint

SHARED COMPONENT LIBRARY (reuse across all screens)

- Persistent left sidebar navigation (Dashboard, Live Monitoring, Crowd Intelligence, Incident Alerts, Workforce Safety, Analytics, Settings) with active-state highlighting
- Top navigation bar: station selector, shift/time indicator, connection status, notification bell with live alert count, user/role badge
- KPI summary cards with large numerals, trend delta, and status color coding
- Reusable risk/status indicator (Low / Medium / High / Critical) as a consistent pill or chip component used across every screen
- Interactive line/bar charts, density heatmaps, and a mini station map component
- Alert panel component (icon, title, location, camera ID, confidence %, timestamp, status, recommended action) reused in both the dashboard and the alert center
- Data tables with sortable columns, pagination, and status chips
- Horizontal activity timeline component for incident history

SCREEN 1 – Railway Command Center Dashboard

Primary landing screen for control-room operators. Include:

- KPI cards: Total Connected Cameras, Active Incidents, Crowd Risk Status, Station Safety Score
- Live Alerts feed (compact, scrollable, most recent first)
- Analytics summary panel (incident trend over time, detection accuracy)
- Platform-wise monitoring grid showing each platform's live status
- Station map widget with camera pins colour-coded by status
- Activity timeline strip along the bottom summarising the last 24 hours

SCREEN 2 – Live CCTV Monitoring

Represents real-time multi-camera surveillance. Include:

- Multi-camera grid (4/9/16 view toggle) with camera location label per feed
- AI detection overlays: bounding boxes around detected persons/objects
- Detection label + confidence percentage rendered on each bounding box
- Incident markers overlaid on the relevant feed the moment they are raised
- Camera control strip: feed selection, zoom, and camera health indicator

SCREEN 3 – Crowd Intelligence Dashboard

Include:

- Crowd density heatmap overlaid on a platform floor-plan view
- Per-platform crowd level readout using Low / Medium / High / Critical states
- Passenger flow analysis chart (entries vs exits over time)
- Congestion prediction panel (short-horizon forecast trend line)
- Queue monitoring widget for ticketing/security-check areas

SCREEN 4 – Smart Incident Alert Center

Include a scrollable list of incident cards, each structured as:

Incident: Unattended Object | Location: Platform 4 | Camera: CAM_042

Confidence: 96% | Status: Security Team Notified

Each card must also show: alert priority (color-coded), timestamp, a camera evidence thumbnail, and a recommended-action line. Include filter controls by priority, status, and camera zone, plus a detail drawer that opens on card click.

SCREEN 5 – Workforce Safety Monitoring

Include:

- Worker activity feed (role, zone, current activity)
- PPE compliance panel (helmet/vest detection status per worker, compliance %)
- Restricted zone entry detection with live alert state
- Safety violation log with severity and timestamp
- Work progress tracker for active maintenance/track-work zones

DELIVERABLE

Generate all five screens as a connected prototype sharing one design system, one sidebar, and one top navigation bar, with realistic railway-operations sample data populated throughout (station names, platform numbers, camera IDs, timestamps).

Antigravity Prompt

Copy the full block below into Antigravity as a single generation prompt. It defines the system architecture, technology stack, MVP feature scope, database schema, and REST API surface required for a hackathon-realistic but production-minded implementation.

PROMPT — PASTE INTO ANTIGRAVITY

ROLE & OBJECTIVE

Act as a senior full-stack solution architect. Design and scaffold a functional MVP for "VisionGuard AI", an AI-powered CCTV intelligence platform for Indian Railways (SIH1349). Think like a senior engineering team building toward a real government deployment – prioritise realistic hackathon-buildable scope over speculative features.

HIGH-LEVEL ARCHITECTURE (implement in this order)

Existing CCTV Cameras

-> RTSP/IP Video Streams

-> Video Processing Layer (OpenCV + frame sampling/pre-processing)

-> AI Vision Intelligence Layer:

Person Detection | Person Tracking | Crowd Density Analysis |

Activity Recognition | PPE Detection | Fire/Smoke Detection

-> Decision Intelligence Layer: risk scoring, event classification, confidence filtering, alert prioritisation

-> Notification Layer (alert routing to security/ops teams)

-> Dashboard Application (control-room UI, consumes REST + live feeds)

TECHNOLOGY STACK

- Frontend: React.js + Tailwind CSS

- Backend: FastAPI (Python)

- AI/ML: YOLOv11 (detection), ByteTrack (multi-object tracking), OpenCV (video processing), PyTorch (model runtime), VideoMAE (activity recognition)

- Database: PostgreSQL

- Deployment: Docker containers, GPU acceleration for inference, designed to extend to edge-AI deployment at station level

MVP FEATURE SCOPE

1. CCTV Simulation Module – camera list, simulated live video feed per camera, per-camera online/offline status (stand-in for real RTSP feeds in the demo)
2. Crowd Monitoring Module – person counting, density calculation, heatmap generation, aggregate crowd risk score per platform/zone
3. Incident Detection Module – detect abandoned objects, restricted-area entry, and suspicious activity; generate an alert with evidence frame and confidence score for each detection
4. Workforce Monitoring Module – PPE detection, safety compliance scoring, worker activity classification
5. Command Dashboard – real-time alert feed, analytics panels, historical incident reports, incident history log

DATABASE SCHEMA

Camera(camera_id, location, status, stream_url)

Incident(incident_id, type, severity, confidence, timestamp)

Alert(alert_id, incident_id [FK -> Incident], status, assigned_team)

Worker(worker_id, activity, compliance_status)

CrowdLog(camera_id [FK -> Camera], density, risk_score, timestamp)

User(user_id, role, permissions)

Design all foreign-key relationships explicitly and add indexes on timestamp columns used for time-series analytics queries.

REST API SURFACE

GET /cameras -> list all cameras and live status

GET /live-feed -> stream/reference active camera feed metadata

GET /alerts -> list active + historical alerts (filterable)

GET /analytics -> aggregated crowd, incident, and compliance analytics

POST /incident -> register a newly detected incident

POST /notify -> trigger notification/routing to the assigned team

ENGINEERING PRINCIPLES TO DEMONSTRATE

- Scalability: architecture must scale from a single station, to multiple stations, to a smart-city-wide deployment without redesign
- Privacy: implement face masking on stored/exported footage, role-based access control (RBAC), secure storage of video evidence, and audit logs for every access/action on sensitive data
- Reliability: reduce false alerts via confidence-threshold filtering and multi-frame verification before an alert is raised
- Production thinking: structure the codebase (services, workers, API, frontend) as a deployable system, not a single notebook/script

DELIVERABLE
Scaffold the repository structure, the FastAPI service with the routes above, the PostgreSQL schema/migrations, the AI inference pipeline stubs for each module, and a React+Tailwind dashboard shell wired to the API – ready for the UI generated from the companion Google Stitch prompt to be integrated in.

Implementation Reference Summary

Quick-reference tables for the engineering team while building against the two prompts above.

Technology Stack

Layer	Technology
Frontend	React.js, Tailwind CSS
Backend	FastAPI (Python)
AI / ML	YOLOv11, ByteTrack, OpenCV, PyTorch, VideoMAE
Database	PostgreSQL
Deployment	Docker, GPU acceleration, Edge AI-ready

Database Tables

Table	Fields
Camera	camera_id, location, status, stream_url
Incident	incident_id, type, severity, confidence, timestamp
Alert	alert_id, incident_id, status, assigned_team
Worker	worker_id, activity, compliance_status
CrowdLog	camera_id, density, risk_score, timestamp
User	user_id, role, permissions

REST API Endpoints

Endpoint	Purpose
GET /cameras	List all cameras and live status
GET /live-feed	Active camera feed metadata / stream reference
GET /alerts	Active and historical alerts, filterable
GET /analytics	Aggregated crowd, incident, and compliance analytics
POST /incident	Register a newly detected incident
POST /notify	Route notification to the assigned response team

Scalability & Security Principles

- **Scalability** — architecture supports single station → multiple stations → smart-city deployment
- **Privacy** — face masking, role-based access control, secure storage, full audit logs
- **Reliability** — false-alert reduction via confidence filtering and multi-frame verification
- **Production thinking** — services structured for a real government deployment, not a demo script